MODUL PRAKTIIKUM 03

STATE MANAGEMENT DAN INTERAKSI PENGGUNA

Pengantar State Management • setState() untuk Update UI
TextField • Checkbox • RadioButton
ElevatedButton • FloatingActionButton • GestureDetector



Mata Kuliah	Pemrograman Perangkat Bergerak Lanjut
Modul	III – State Management dan Interaksi Pengguna
Tahun	2026

Modul praktikum ini digunakan sebagai panduan kegiatan praktikum mahasiswa.

DAFTAR ISI

A. Tujuan Pembelajaran 3
B. Tugas Pendahuluan 3
C. Dasar Teori 3
a. State dan Hubungannya dengan UI 4
b. StatelessWidget dan StatefulWidget 4
c. State Lokal dan State Aplikasi 5
d. Menggunakan setState() 5
e. Kapan setState() Mulai Sulit Dikelola 6
f. ChangeNotifier dan Provider 6
g. watch, read, select, dan Consumer 7
h. Memilih setState() atau Provider 8
D. Alat dan Bahan 8
a. Perangkat Keras (Hardware) 8
b. Perangkat Lunak (Software) 8
E. Materi Praktikum 8
a. Menyiapkan Proyek Praktikum 9
b. Membuat State Lokal dengan setState() 9
c. Menjalankan dan Mengamati setState() 11
d. Membuat Model Keranjang dengan ChangeNotifier 12
e. Menyediakan Model dengan ChangeNotifierProvider 14
f. Membuat Halaman Produk dengan context.read() 16
g. Membuat Halaman Keranjang dengan Consumer 17
h. Menggabungkan setState() dan Provider 19
F. Materi Praktikum dengan Prompt 22
a. Menyiapkan Proyek Praktikum dengan Bantuan AI 23
b. Membuat Latihan setState() dengan Bantuan AI 24
c. Memahami Alur setState() dengan Bantuan AI 25
d. Merancang Model Keranjang dengan Bantuan AI 26
e. Membuat ChangeNotifier dengan Bantuan AI 27
f. Memasang Provider pada Akar Aplikasi dengan Bantuan AI 28
g. Membuat Halaman Produk dengan Bantuan AI 29
h. Membuat Halaman Keranjang dengan Bantuan AI 30
i. Menggabungkan, Menguji, dan Memperbaiki Galat dengan Bantuan AI 31
j. Mengevaluasi Jawaban AI 32
G. Tugas Praktikum 32
H. Format Pengumpulan Tugas 35

A. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan state sebagai data yang dapat berubah dan memengaruhi tampilan antarmuka.
- Membedakan StatelessWidget dan StatefulWidget berdasarkan kebutuhan perubahan data.
- Membedakan state lokal (ephemeral state) dan state aplikasi (app state) dengan contoh sederhana.
- Mengubah state lokal melalui setState() sehingga Flutter menjadwalkan pembangunan ulang UI.
- Membuat model state menggunakan ChangeNotifier dan memberi tahu pendengar melalui notifyListeners().
- Menyediakan model dengan ChangeNotifierProvider serta membacanya melalui context.watch, context.read, context.select, atau Consumer.
- Memilih setState() atau Provider berdasarkan luas pemakaian data, bukan karena salah satunya dianggap selalu lebih baik.
- Menguji perubahan state dan menelusuri galat umum, seperti UI tidak berubah atau ProviderNotFoundException.

B. Tugas Pendahuluan

1. Apa yang dimaksud dengan state pada Flutter, dan jelaskan hubungan antara perubahan state dengan pembangunan ulang (rebuild) antarmuka pengguna!
2. Jelaskan perbedaan antara StatelessWidget dan StatefulWidget, serta berikan contoh kondisi yang tepat untuk menggunakan masing-masing widget tersebut!
3. Apa perbedaan antara state lokal (ephemeral state) dan state aplikasi (app state)? Berikan masing-masing satu contoh, serta jelaskan kapan sebaiknya memilih setState() dan kapan memilih Provider!
4. Jelaskan cara kerja setState() dalam memperbarui tampilan, serta sebutkan kondisi yang membuat penggunaan setState() mulai sulit dikelola!
5. Apa yang dimaksud dengan ChangeNotifier dan ChangeNotifierProvider, serta jelaskan fungsi pemanggilan notifyListeners() dalam pengelolaan state menggunakan Provider!
6. Jelaskan perbedaan penggunaan context.watch(), context.read(), context.select(), dan widget Consumer dalam membaca data dari Provider!

C. Dasar Teori

Antarmuka Flutter dibangun dari data. Ketika data berubah, tampilan yang memakai data tersebut perlu dibangun ulang. Data yang dapat berubah selama aplikasi berjalan disebut state. Contohnya adalah jumlah barang, status tombol aktif, tab yang dipilih, teks hasil pencarian, dan data pengguna yang sudah masuk. Agar lebih mudah dibayangkan, state dapat dianggap seperti papan skor pertandingan: angkanya bisa berubah kapan saja, dan setiap kali berubah, penonton (tampilan aplikasi) harus segera melihat angka yang terbaru, bukan angka yang lama.

> **Kalimat paling sederhana:** State adalah ingatan sementara yang dipakai aplikasi untuk menentukan apa yang harus terlihat di layar saat ini.

a. State dan Hubungannya dengan UI

Flutter memakai pendekatan deklaratif. Pengembang tidak memerintahkan layar untuk mengganti satu label secara langsung. Pengembang mengubah data, kemudian Flutter menjalankan kembali metode build() untuk menghasilkan tampilan yang sesuai dengan data terbaru. Pendekatan ini mirip dengan memesan makanan di restoran: pelanggan cukup menyebutkan menu yang diinginkan, dan koki (Flutter) yang menentukan cara menyajikannya, bukan pelanggan yang masuk ke dapur untuk mengubah masakan secara langsung. Dengan kata lain, build() adalah instruksi "gambarkan ulang layar sesuai data yang ada sekarang", bukan perintah untuk mengubah satu bagian layar secara manual.



Gambar 2.1 Alur sederhana perubahan state pada Flutter

Jika digambarkan sebagai diagram alur, urutannya adalah: Aksi pengguna → State berubah → build() dipanggil → Tampilan pada layar diperbarui. Metode build() boleh dipanggil berkali-kali. Karena itu, build() sebaiknya hanya menyusun widget dan tidak melakukan pekerjaan berat, membaca berkas besar, atau mengubah state secara terus-menerus.

b. StatelessWidget dan StatefulWidget

StatelessWidget cocok ketika data tampilan tidak berubah selama widget tersebut hidup. StatefulWidget digunakan ketika satu layar perlu menyimpan data yang dapat berubah. StatefulWidget sendiri tetap bersifat tetap; data yang berubah disimpan di dalam objek State yang dibuat melalui createState(). Perumpamaan sederhananya, StatelessWidget seperti foto cetak yang sekali jadi tidak berubah lagi, sedangkan StatefulWidget seperti bingkai yang menempel pada papan tulis kecil bernama objek State, tempat data boleh dihapus dan ditulis ulang selama layar itu masih dipakai. Tabel berikut membantu menjawab pertanyaan "widget mana yang harus dipakai" sebelum kode mulai ditulis.

Tabel 2.1 Perbandingan StatelessWidget dan StatefulWidget

Pertanyaan	Gunakan StatelessWidget	Gunakan StatefulWidget
Apakah data berubah?	Tidak berubah dari dalam widget	Dapat berubah selama layar digunakan
Contoh	Logo, judul, kartu informasi tetap	Counter, pilihan tab, status sakelar

Pertanyaan	Gunakan StatelessWidget	Gunakan StatefulWidget
Tempat data	Nilai final dari konstruktor	Variabel di dalam kelas State
Cara memperbarui UI	Data baru diberikan dari widget induk	Panggil setState() untuk state lokal

c. State Lokal dan State Aplikasi

Tidak semua state harus ditempatkan di Provider. Pilihan alat ditentukan oleh siapa yang membutuhkan data dan berapa lama data tersebut perlu hidup. Perumpamaannya, state lokal seperti catatan kecil di meja pribadi yang hanya dibaca pemiliknya sendiri, sedangkan state aplikasi seperti papan pengumuman di lorong yang bisa dibaca banyak orang dari ruangan berbeda. Sebelum memilih tools, berikut merupakan perbedaan antaraState lokal dan State Aplikasi seabgaimana Tabel berikut.

Tabel 2.2 Perbandingan State Lokal dan State Aplikasi

Jenis state	Bahasa mudah	Contoh	Alat awal
State lokal (ephemeral)	Hanya penting bagi satu widget atau satu layar.	Indeks NavigationBar, sakelar tampil/sembunyi, animasi lokal.	setState()
State aplikasi (app state)	Dipakai bersama oleh beberapa bagian aplikasi.	Keranjang belanja, pengguna yang masuk, tema aplikasi.	Provider

d. Menggunakan setState()

setState() adalah metode milik objek State. Perubahan variabel ditulis di dalam callback setState(). Setelah callback selesai, Flutter menandai bagian widget tersebut agar dibangun ulang. Callback di sini berarti kumpulan kode yang dituliskan di dalam kurung kurawal setelah setState(), yang langsung dijalankan begitu setState() dipanggil. Bayangkan setState() seperti menekan bel di meja resepsionis sambil menyerahkan catatan perubahan; begitu bel ditekan, resepsionis (Flutter) segera memperbarui papan informasi (tampilan layar) sesuai catatan tersebut.

Program inti setState()

```
setState(() {
  jumlah++;
});
```

- Callback setState() harus berjalan cepat dan tidak boleh diberi kata kunci async.
- Ubah hanya data yang benar-benar diperlukan, lalu biarkan build() membaca nilai terbarunya.
- Jangan memanggil setState() dari dalam build(), karena hal itu dapat membuat pembangunan ulang berulang tanpa selesai.

Pola aman setelah proses asynchronous

```
final data = await ambilData();
if (!mounted) return;
setState(() {
  hasil = data;
});
```

e. Kapan setState() Mulai Sulit Dikelola

setState() bukan alat yang salah. Alat ini menjadi kurang nyaman ketika data dan aturan bisnis tersebar di banyak layar. Gejalanya terlihat ketika callback harus diteruskan melewati banyak widget, beberapa layar memerlukan data yang sama, atau satu kelas State berisi terlalu banyak aturan. Kondisi ini mirip permainan pesan berantai: semakin banyak widget yang harus meneruskan data dari satu ke widget lain, semakin besar peluang data tersebut membingungkan atau tercecer di tengah jalan. Empat tanda berikut membantu mengenali saat yang tepat untuk mulai berpindah ke Provider.

- Data dipakai oleh dua halaman atau lebih.
- Banyak widget membutuhkan nilai yang sama pada kedalaman pohon yang berbeda.
- Logika tambah, hapus, hitung total, dan validasi bercampur dengan kode tampilan.
- Pengujian logika sulit dilakukan tanpa membuat widget.

f. ChangeNotifier dan Provider

ChangeNotifier adalah kelas dari Flutter SDK yang dapat memberi tahu pendengar ketika datanya berubah. Provider adalah paket yang membantu menempatkan objek pada pohon widget dan mengambilnya kembali dari widget keturunan. Pada pola sederhana, ChangeNotifier menyimpan data dan aturan, sedangkan ChangeNotifierProvider mengatur pembuatan serta masa hidup objek tersebut. Supaya lebih mudah dibayangkan, anggap ChangeNotifier sebagai stasiun radio kecil yang menyiarkan pengumuman setiap kali ada info baru, dan notifyListeners() adalah tombol yang ditekan penyiar untuk mulai menyiarkan pengumuman itu. Widget yang "mendengarkan" (listener) akan langsung tahu ada info baru dan memperbarui tampilannya sendiri. Provider berperan seperti memasang antena bersama pada sebuah gedung bertingkat, yaitu pohon widget (susunan widget yang bertingkat-tingkat seperti struktur folder di dalam folder): sekali antena dipasang di lantai atas, dekat akar aplikasi, semua ruangan atau widget di bawahnya dapat menangkap siaran yang sama tanpa kabel terpisah untuk tiap ruangan. Alur singkatnya dapat digambarkan sebagai: Model diubah → notifyListeners() dipanggil → Provider meneruskan kabar → widget yang berlangganan (watch, select, atau Consumer) membangun ulang tampilannya.

Tabel 2.3 Bagian-Bagian ChangeNotifier dan Provider

Bagian	Tugasnya	Kata kunci
Model	Menyimpan state dan menyediakan operasi untuk mengubahnya.	extends ChangeNotifier
Pemberi	Menempatkan model di atas widget yang membutuhkannya.	ChangeNotifierProvider

Bagian	Tugasnya	Kata kunci
Pemberitahuan	Memberi tahu pendengar bahwa data sudah berubah.	notifyListeners()
Pembaca	Membaca nilai atau memanggil operasi dari UI.	watch, read, select, Consumer

Bentuk paling kecil sebuah ChangeNotifier

```
class CounterModel extends ChangeNotifier {
  int nilai = 0;

  void tambah() {
    nilai++;
    notifyListeners();
  }
}
```

g. watch, read, select, dan Consumer

Keempat cara berikut mengambil model yang disediakan Provider. Perbedaannya terletak pada apakah widget harus dibangun ulang dan seberapa luas bagian yang mendengarkan perubahan. context.watch() seperti menyalakan radio dan terus mendengarkan setiap pengumuman baru; context.read() seperti menengok radio sekali untuk mengecek info lalu langsung mematikannya tanpa terus mendengarkan; context.select() seperti hanya memasang telinga pada satu segmen siaran tertentu dan mengabaikan segmen lain; sedangkan Consumer seperti memasang pengeras suara kecil hanya di satu sudut ruangan, sehingga hanya sudut itu yang ramai setiap kali ada pengumuman baru.

Tabel 2.4 Perbandingan watch, read, select, dan Consumer

Cara	Apa yang terjadi	Kapan dipakai	Contoh inti
context.watch<T>()	Widget mendengarkan T dan build() dipanggil ketika T memberi notifikasi.	Saat tampilan membaca beberapa nilai model.	final model = context.watch<CounterModel>();
context.read<T>()	Mengambil T sekali tanpa mendengarkan perubahan.	Di callback tombol untuk memanggil metode.	context.read<CounterModel>().tambah();
context.select<T, R>()	Hanya mendengarkan hasil R yang dipilih.	Saat widget cukup bergantung pada satu nilai kecil.	final nilai = context.select<CounterModel, int>((m) => m.nilai);
Consumer<T>	Builder di dalam Consumer dibangun ulang ketika T berubah.	Saat area yang mendengar perubahan ingin dibuat jelas.	Consumer<CounterModel>(builder: ...)

Kesalahan umum: context.read() tidak membuat widget mendengarkan perubahan. Jika read() dipakai untuk menampilkan nilai di build(), layar tidak otomatis berubah. Gunakan watch(), select(), atau Consumer untuk bagian yang harus mengikuti state.

h. Memilih setState() atau Provider

Tabel 2.5 Panduan Memilih setState() atau Provider

Pertanyaan	Jika jawabannya YA	Pilihan awal
Apakah data hanya dipakai satu widget atau satu layar?	Contoh: indeks tab lokal.	setState()
Apakah data harus dibaca beberapa halaman?	Contoh: isi keranjang.	Provider
Apakah logika perlu diuji tanpa UI?	Simpan aturan di model terpisah.	Provider + ChangeNotifier
Apakah perubahan hanya untuk animasi atau kontrol tampilan sesaat?	Biarkan tetap dekat dengan widget.	setState()

D. Alat dan Bahan

a. Perangkat Keras (Hardware)

- Komputer atau laptop 64 bit dengan RAM minimal 8 GB; 16 GB disarankan apabila emulator dan Android Studio dijalankan bersamaan.
- Perangkat Android fisik dan kabel data sebagai alternatif emulator, dengan USB Debugging yang telah aktif.

b. Perangkat Lunak (Software)

- Flutter SDK pada kanal stabil dan Dart SDK yang dikenali oleh perintah flutter doctor.
- Android Studio atau Visual Studio Code beserta ekstensi Flutter dan Dart.
- Paket provider. Tambahkan melalui perintah flutter pub add provider agar versi yang sesuai dicatat otomatis pada pubspec.yaml.
- Git dan akun repositori untuk menyimpan hasil praktikum.

E. Materi Praktikum

Praktikum memakai satu cerita sederhana: aplikasi Kantin Kampus. Mahasiswa lebih dahulu melihat state lokal melalui halaman penghitung pesanan. Setelah itu, aplikasi dikembangkan menjadi dua halaman yang berbagi keranjang melalui Provider. Singkatnya, praktikum ini berjalan dalam dua babak: babak pertama berlatih setState() sendirian pada satu layar sederhana, lalu babak kedua menyambungkan dua layar (Produk dan Keranjang) memakai Provider agar perbedaan antara state lokal dan state aplikasi.

a. Menyiapkan Proyek Praktikum

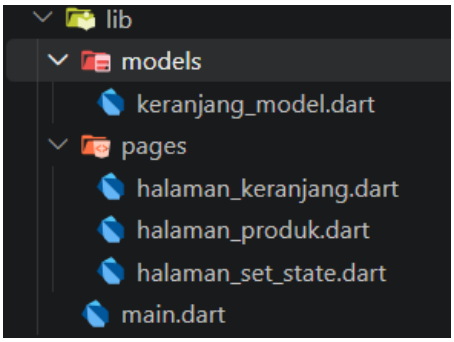
- Buka terminal, lalu buat proyek baru. Ganti nim dengan nomor induk masing-masing.

Perintah terminal

```
flutter create state_interaksi_nim
cd state_interaksi_nim
flutter pub add provider
flutter run
```

- Buat folder models dan pages di dalam lib. Folder models nantinya menyimpan berkas yang mengatur data (seperti keranjang belanja), sedangkan folder pages menyimpan setiap layar/tampilan aplikasi; memisahkan keduanya membantu kode tetap rapi karena berkas di models fokus mengurus data dan aturan, sementara berkas di pages fokus menggambar tampilan. Struktur akhir yang akan dibuat adalah sebagai berikut.

Struktur berkas



Gambar 4.1 Struktur Berkas

Pemeriksaan: Buka pubspec.yaml dan pastikan provider tercatat di bawah dependencies, bukan dev_dependencies.

b. Membuat State Lokal dengan setState()

- Buat berkas lib/pages/halaman_set_state.dart, kemudian tuliskan kode berikut.

lib/pages/halaman_set_state.dart

```
// lib/pages/halaman_set_state.dart
import 'package:flutter/material.dart';

class HalamanSetState extends StatefulWidget {
  const HalamanSetState({super.key});

  @override
  State<HalamanSetState> createState() => _HalamanSetStateState();
}

class _HalamanSetStateState extends State<HalamanSetState> {
  int _jumlahPesanan = 0;
  bool _kantinBuka = true;
```

```
void _tambahPesanan() {
  setState(() {
    _jumlahPesanan++;
  });
}

void _kurangiPesanan() {
  if (_jumlahPesanan == 0) return;

  setState(() {
    _jumlahPesanan--;
  });
}

void _ubahStatusKantin(bool nilaiBaru) {
  setState(() {
    _kantinBuka = nilaiBaru;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Latihan setState'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Card(
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Column(
                children: [
                  const Text('Jumlah pesanan hari ini'),
                  Text(
                    '$_jumlahPesanan',
                    style: Theme.of(context).textTheme.displayMedium,
                  ),
                  Row(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      IconButton(
                        onPressed: _kurangiPesanan,
                        icon: const Icon(Icons.remove_circle_outline),
                      ),
                      const SizedBox(width: 12),
                      FilledButton.icon(
                        onPressed: _tambahPesanan,
```

```
                    icon: const Icon(Icons.add),
                    label: const Text('Tambah'),
                  ),
                ],
              ),
            ],
          ),
        ),
      ),
      const SizedBox(height: 16),
      SwitchListTile(
        title: const Text('Kantin buka'),
        subtitle: Text(
          _kantinBuka ? 'Menerima pesanan' : 'Sedang tutup',
        ),
        value: _kantinBuka,
        onChanged: _ubahStatusKantin,
      ),
    ],
  ),
 ),
);
 }
}
```

- Variabel _jumlahPesanan dan _kantinBuka adalah state lokal karena hanya dipakai oleh halaman ini. Nama variabel yang diawali garis bawah (_) menandakan variabel privat, yaitu hanya bisa diakses dari dalam berkas ini sendiri.
- Setiap fungsi mengubah nilai di dalam setState(). Setelah itu, Flutter memanggil kembali build() dan teks pada layar mengikuti nilai terbaru. Ingat analogi bel resepsionis pada Dasar Teori: memanggil setState() sama seperti menekan bel sambil menyerahkan catatan perubahan, dan Flutter langsung memperbarui tampilan sesuai catatan itu.
- Pemeriksaan if (_jumlahPesanan == 0) mencegah jumlah menjadi negatif.

c. Menjalankan dan Mengamati setState()

- Ganti sementara isi lib/main.dart dengan kode berikut agar halaman latihan menjadi halaman pertama.

lib/main.dart - versi sementara

```
// lib/main.dart - sementara untuk menguji setState
import 'package:flutter/material.dart';

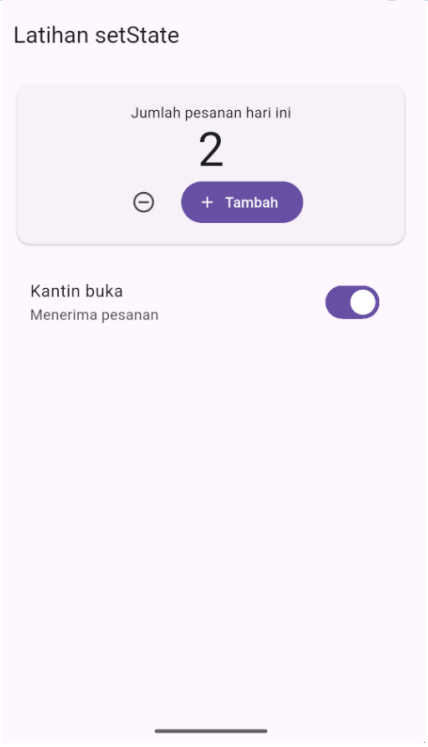
import 'pages/halaman_set_state.dart';

void main() {
 runApp(
   const MaterialApp(
     debugShowCheckedModeBanner: false,
```

```
      home: HalamanSetState(),
    ),
  );
}
```

Tabel 4.1 Penjelasan Program setState()

Aksi	State yang berubah	Tampilan yang harus terlihat
Tekan Tambah	_jumlahPesanan bertambah 1	Angka pada kartu bertambah
Tekan Kurangi	_jumlahPesanan berkurang 1	Angka tidak pernah kurang dari 0
Ubah Switch	_kantinBuka menjadi true/false	Subjudul berubah antara menerima pesanan dan tutup



Gambar 4.2 SetState

d. Membuat Model Keranjang dengan ChangeNotifier

Keranjang diperlukan oleh halaman produk dan halaman keranjang. Karena data dipakai bersama, logika ditempatkan dalam KeranjangModel. UI hanya memanggil operasi tambah, kurangi, dan kosongkan. Di sinilah state aplikasi (app state) yang dibahas pada Dasar Teori benar-benar dipakai: KeranjangModel menjadi satu-satunya sumber data yang dibaca oleh dua halaman berbeda, bukan disalin sendiri-sendiri oleh tiap halaman.

- Buat berkas lib/models/keranjang_model.dart, kemudian tuliskan kode berikut.

lib/models/keranjang_model.dart

```
// lib/models/keranjang_model.dart
import 'package:flutter/foundation.dart';

@immutable
class Produk {
  const Produk({
    required this.nama,
    required this.harga,
    required this.ikon,
  });

  final String nama;
  final int harga;
  final String ikon;
}

class ItemKeranjang {
  ItemKeranjang({required this.produk, this.jumlah = 1});

  final Produk produk;
  int jumlah;
}

class KeranjangModel extends ChangeNotifier {
  final Map<String, ItemKeranjang> _items = {};

  List<ItemKeranjang> get items => List.unmodifiable(_items.values);

  int get totalItem {
    return _items.values.fold(0, (total, item) => total + item.jumlah);
  }

  int get totalHarga {
    return _items.values.fold(
      0,
      (total, item) => total + (item.produk.harga * item.jumlah),
    );
  }

  void tambah(Produk produk) {
    final itemLama = _items[produk.nama];

    if (itemLama == null) {
      _items[produk.nama] = ItemKeranjang(produk: produk);
    } else {
      itemLama.jumlah++;
    }

    notifyListeners();
  }
```

```
  void kurangi(Produk produk) {
    final itemLama = _items[produk.nama];
    if (itemLama == null) return;

    if (itemLama.jumlah == 1) {
      _items.remove(produk.nama);
    } else {
      itemLama.jumlah--;
    }

    notifyListeners();
  }

  void kosongkan() {
    if (_items.isEmpty) return;
    _items.clear();
    notifyListeners();
  }
}
```

- Daftar internal _items dibuat privat agar UI tidak dapat mengubah isinya secara sembarangan.
- Getter items, totalItem, dan totalHarga menyediakan hasil yang aman untuk dibaca UI. Getter adalah fungsi yang dipanggil seperti variabel biasa (tanpa tanda kurung) dan selalu menghitung ulang nilainya saat diakses, sehingga data yang dibaca UI selalu mengikuti kondisi _items yang terbaru.
- Setiap operasi yang benar-benar mengubah data diakhiri dengan notifyListeners(). Tanpa pemanggilan ini, widget yang mendengarkan model tidak mengetahui perubahan. Seperti analogi stasiun radio pada Dasar Teori, notifyListeners() adalah tombol siaran; lupa memanggilnya sama seperti penyiar yang sudah menulis pengumuman tetapi lupa menekan tombol siaran, sehingga pendengar (widget) tidak pernah tahu ada info baru.

e. Menyediakan Model dengan ChangeNotifierProvider

Provider harus ditempatkan di atas seluruh halaman yang memerlukan KeranjangModel. Pada versi akhir, ChangeNotifierProvider membungkus AplikasiKantin sehingga halaman produk dan halaman keranjang berada di bawah provider yang sama.

lib/main.dart (menguji Provider)

```
// lib/main.dart - sementara untuk menguji Provider
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'models/keranjang_model.dart';

void main() {
  runApp(
```

```
    ChangeNotifierProvider(
      create: (context) => KeranjangModel(),
      child: const ContohProviderApp(),
    ),
  );
}

class ContohProviderApp extends StatelessWidget {
  const ContohProviderApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Uji Coba Provider')),
        body: Center(
          child: Consumer<KeranjangModel>(
            builder: (context, keranjang, child) {
              keranjang.tambah(Produk(nama: 'peroduk', harga: 15000,
ikon: ''));
              return Text(
                'Jumlah item di keranjang: ${keranjang.totalItem}',
                style: const TextStyle(fontSize: 20),
              );
            },
          ),
        ),
      ),
    );
  }
}
```



Gambar 4.3 Ujicoba ChangeNotifierProvider

f. Membuat Halaman Produk dengan context.read()

Halaman produk tidak perlu dibangun ulang ketika keranjang berubah karena isi daftar produknya tetap. Tombol Tambah hanya memanggil sebuah operasi. Karena itu, context.read() tepat digunakan di dalam callback onPressed. Mengacu pada perumpamaan radio di Dasar Teori, tombol Tambah di sini berperilaku seperti context.read(): ia hanya "menengok" KeranjangModel sekali untuk menjalankan perintah tambah(), tanpa perlu terus mendengarkan perubahan apa pun setelahnya.

lib/pages/halaman_produk.dart

```
// lib/pages/halaman_produk.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/keranjang_model.dart';

class HalamanProduk extends StatelessWidget {
 const HalamanProduk({super.key});

 static const daftarProduk = [
  Produk(nama: 'Nasi Goreng', harga: 15000, ikon: '🍛'),
  Produk(nama: 'Mie Ayam', harga: 13000, ikon: '🍜'),
  Produk(nama: 'Es Teh', harga: 5000, ikon: '🧋'),
 ];

 String rupiah(int nilai) {
  return 'Rp${nilai.toString()}';
 }

 @override
 Widget build(BuildContext context) {
  return ListView.separated(
   padding: const EdgeInsets.all(16),
   itemCount: daftarProduk.length,
   separatorBuilder: (context, indeks) => const SizedBox(height: 8),
   itemBuilder: (context, indeks) {
    final produk = daftarProduk[indeks];

    return Card(
     child: ListTile(
      leading: Text(
       produk.ikon,
       style: const TextStyle(fontSize: 32),
      ),
      title: Text(produk.nama),
      subtitle: Text(rupiah(produk.harga)),
      trailing: FilledButton(
       onPressed: () {
        context.read<KeranjangModel>().tambah(produk);
       },
```

```
            child: const Text('Tambah'),
          ),
        ),
      );
    },
  );
 }
}
```

- Import package:provider/provider.dart diperlukan agar extension read() dikenali pada BuildContext. Extension adalah cara Dart menambahkan kemampuan baru pada tipe data yang sudah ada; paket provider menambahkan kemampuan .read(), .watch(), dan .select() pada BuildContext lewat mekanisme ini.
- context.read<KeranjangModel>() mencari KeranjangModel terdekat di atas widget, kemudian metode tambah() dijalankan.
- read() tidak membuat seluruh daftar produk dibangun ulang setiap kali isi keranjang berubah.

g. Membuat Halaman Keranjang dengan Consumer

Halaman keranjang harus mengikuti setiap perubahan item. Consumer<KeranjangModel> membangun ulang bagian di dalam builder ketika notifyListeners() dipanggil. Sesuai analogi pengeras suara pada Dasar Teori, Consumer<KeranjangModel> di sini bertindak seperti pengeras suara yang dipasang khusus di halaman keranjang: hanya bagian di dalam builder yang ramai (dibangun ulang) setiap kali ada pengumuman baru dari KeranjangModel, tanpa memengaruhi bagian lain di luar Consumer.

lib/pages/halaman_keranjang.dart

```
// lib/pages/halaman_keranjang.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/keranjang_model.dart';

class HalamanKeranjang extends StatelessWidget {
 const HalamanKeranjang({super.key});

 String rupiah(int nilai) {
  return 'Rp${nilai.toString()}';
 }

 @override
 Widget build(BuildContext context) {
  return Consumer<KeranjangModel>(
   builder: (context, keranjang, child) {
    if (keranjang.items.isEmpty) {
     return const Center(
      child: Column(
```

```
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(Icons.shopping_cart_outlined, size: 64),
          SizedBox(height: 12),
          Text('Keranjang masih kosong'),
        ],
      ),
    );
  }

  return Column(
    children: [
      Expanded(
        child: ListView.separated(
          padding: const EdgeInsets.all(16),
          itemCount: keranjang.items.length,
          separatorBuilder: (context, indeks) => const Divider(),
          itemBuilder: (context, indeks) {
            final item = keranjang.items[indeks];

            return ListTile(
              title: Text(item.produk.nama),
              subtitle: Text(
                '${item.jumlah} x ${rupiah(item.produk.harga)}',
              ),
              trailing: Row(
                mainAxisSize: MainAxisSize.min,
                children: [
                  IconButton(
                    onPressed: () => keranjang.kurangi(item.produk),
                    icon: const Icon(Icons.remove_circle_outline),
                  ),
                  Text('${item.jumlah}'),
                  IconButton(
                    onPressed: () => keranjang.tambah(item.produk),
                    icon: const Icon(Icons.add_circle_outline),
                  ),
                ],
              ),
            );
          },
        ),
      ),
      SafeArea(
        top: false,
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
```

```
                  'Total: ${rupiah(keranjang.totalHarga)}',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                const SizedBox(height: 8),
                OutlinedButton.icon(
                  onPressed: keranjang.kosongkan,
                  icon: const Icon(Icons.delete_outline),
                  label: const Text('Kosongkan Keranjang'),
                ),
              ],
            ),
          ),
        ),
      ),
    ],
  );
 },
 );
 }
}
```

- Ketika items kosong, halaman menampilkan empty state yang mudah dimengerti pengguna. Empty state adalah istilah umum untuk tampilan saat data masih kosong, biasanya berupa ikon dan pesan singkat agar pengguna tidak melihat layar kosong tanpa keterangan.
- Tombol tambah dan kurang memanggil model yang diterima oleh builder Consumer.
- Total harga adalah getter; nilainya selalu dihitung dari state terbaru, sehingga UI tidak menyimpan salinan total yang mudah tidak sinkron.

h. Menggabungkan setState() dan Provider

Sekarang ganti seluruh isi lib/main.dart dengan versi akhir. Indeks tujuan disimpan lokal melalui setState(), sedangkan jumlah isi keranjang dibaca dari Provider. IndexedStack menjaga keadaan setiap halaman ketika pengguna berpindah tujuan. Bagian ini adalah puncak praktikum: seperti tabel keputusan pada Dasar Teori, _indeks memakai setState() karena hanya penting bagi satu widget (KerangkaUtama), sedangkan totalItem dibaca lewat Provider karena datanya berasal dari KeranjangModel yang dipakai bersama. IndexedStack dipilih agar seluruh halaman tetap hidup di balik layar; jika halaman diganti-ganti dengan cara biasa, state pada halaman yang sedang tidak tampil bisa saja hilang.

lib/main.dart - versi akhir

```
// lib/main.dart - versi akhir dengan Provider
import 'package:flutter/material.dart';
import 'package:provider/provider.dart'; // paket Provider

import 'models/keranjang_model.dart'; // model keranjang (ChangeNotifier)
import 'pages/halaman_keranjang.dart';
import 'pages/halaman_produk.dart';

void main() {
```

```
  runApp(
    ChangeNotifierProvider(
      create: (context) => KeranjangModel(), // KeranjangModel dibuat sekali di
root aplikasi
      child: const AplikasiKantin(),
    ),
  );
}

class AplikasiKantin extends StatelessWidget {
  const AplikasiKantin({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Kantin Kampus',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
        useMaterial3: true,
      ),
      home: const KerangkaUtama(),
    );
  }
}

class KerangkaUtama extends StatefulWidget {
  const KerangkaUtama({super.key});

  @override
  State<KerangkaUtama> createState() => _KerangkaUtamaState();
}

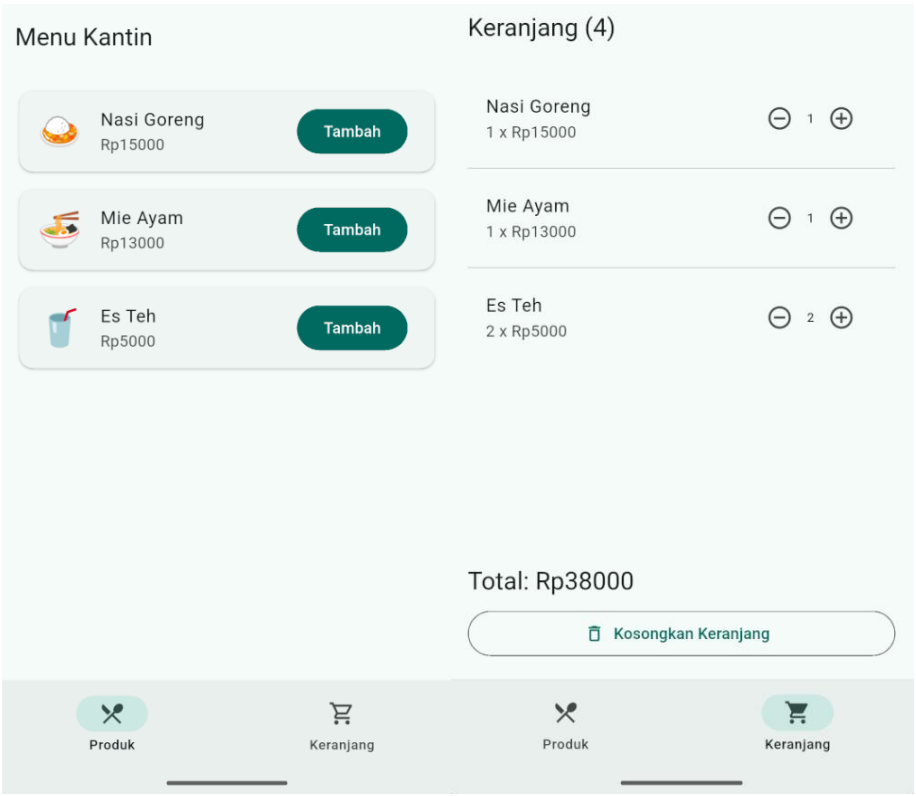
class _KerangkaUtamaState extends State<KerangkaUtama> {
  int _indeks = 0; // state lokal: tetap pakai setState(), bukan Provider

  static const _halaman = [
    HalamanProduk(),
    HalamanKeranjang(),
  ];

  @override
  Widget build(BuildContext context) {
    final totalItem = context.select<KeranjangModel, int>( //baca totalItem dari
Provider
      (model) => model.totalItem,
    );

    return Scaffold(
      appBar: AppBar(
        title: Text(
```

```
          _indeks == 0 ? 'Menu Kantin' : 'Keranjang ($totalItem)',
        ),
      ),
      body: IndexedStack(//setiap halaman tetap hidup di balik layar
        index: _indeks,
        children: _halaman,
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _indeks,
        onDestinationSelected: (nilaiBaru) {
          setState(() { // hanya _indeks yang diperbarui di sini
            _indeks = nilaiBaru;
          });
        },
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.restaurant_menu),
            label: 'Produk',
          ),
          NavigationDestination(
            icon: Icon(Icons.shopping_cart_outlined),
            selectedIcon: Icon(Icons.shopping_cart),
            label: 'Keranjang',
          ),
        ],
      ),
    );
  }
}
```



Gambar 4.4 Aplikasi Kantin Gabungan setState() dan Provider

F. Materi Praktikum dengan Prompt

Bagian ini membangun ulang Materi Praktikum pada bagian E dengan bantuan asisten AI. Hasil akhir yang dituju tetap sama, tetapi kode diperoleh melalui prompt bertahap. Tujuannya bukan menggantikan proses belajar, melainkan melatih mahasiswa menulis kebutuhan dengan jelas, membaca hasil AI, menemukan kesalahan, dan menjelaskan kembali cara kerja kode. Setiap kode dari AI harus dibaca sebelum disalin, dijalankan, dibandingkan dengan ketentuan pada materi sebelumnya, dan diuji. Kode yang belum dapat dijelaskan mahasiswa tidak boleh dianggap selesai.

Tabel 5.1 Unsur prompt yang digunakan pada praktikum berbantuan AI

Unsur prompt	Penjelasan
Peran	Menentukan siapa AI dan tingkat pembacanya, misalnya asisten Flutter untuk mahasiswa pemula.
Konteks	Menjelaskan nama proyek, materi, berkas yang sudah ada, dan tahap yang sedang dikerjakan.
Tugas	Menyebutkan satu hasil yang jelas, misalnya menulis ulang satu berkas tertentu.
Batasan	Menjaga hasil tetap sesuai materi, paket yang diizinkan, nama kelas, dan struktur folder.
Format keluaran	Menentukan apakah AI harus memberi rencana, kode lengkap, bagian yang berubah, atau penjelasan.
Verifikasi	Meminta AI menjelaskan alasan kode dan memberi daftar pemeriksaan yang dapat diuji mahasiswa.

a. Menyiapkan Proyek Praktikum dengan Bantuan AI

Tahap ini menghasilkan proyek baru yang terpisah dari praktikum manual pada bagian E. Dengan demikian, hasil kerja biasa dan hasil berbantuan AI dapat dibandingkan.

- Buka terminal Visual Studio Code, lalu jalankan perintah berikut. Ganti nim dengan nomor induk mahasiswa masing-masing.

```
flutter create state_interaksi_ai_nim
cd state_interaksi_ai_nim
flutter pub add provider
code .
```

- Setelah proyek terbuka, gunakan percakapan AI baru agar konteks dari proyek lain tidak ikut terbawa.
- Ketik prompt konteks awal berikut pada panel percakapan AI.

```
Kamu berperan sebagai asisten pemrograman Flutter untuk mahasiswa pemula.

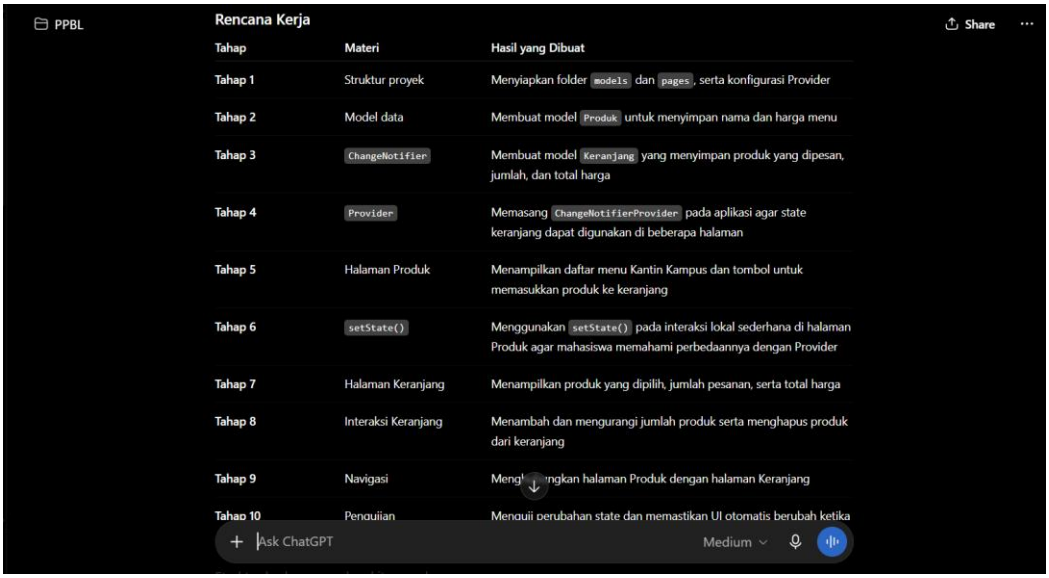
Konteks: saya membuat proyek bernama state_interaksi_ai_nim di Visual Studio Code.
Materi yang dipelajari adalah state, setState(), ChangeNotifier, dan Provider.

Tujuan akhir: membantu saya membuat aplikasi Kantin Kampus dengan halaman Produk dan
halaman Keranjang. Halaman Produk menampilkan daftar menu, sedangkan halaman
Keranjang menampilkan jumlah dan total harga pesanan.

Batasan:
1. Gunakan Flutter Material 3 dan hanya package:flutter/material.dart serta
package:provider/provider.dart.
2. Pisahkan berkas model di lib/models dan berkas halaman di lib/pages.
3. Kerjakan bertahap. Jangan menulis semua kode sekaligus.
4. Setiap kode harus lengkap satu berkas, tidak boleh memakai tanda "..." untuk
menghilangkan bagian kode.
5. Gunakan bahasa Indonesia dan nama kelas yang sama dengan nama berkas.

Format keluaran: tuliskan rencana kerja dan daftar berkas yang akan dibuat. Jangan menulis
kode sebelum saya meminta tahap tertentu.
```

- Bandingkan rencana AI dengan struktur yang dipelajari pada materi di atas. Rencana sekurang-kurangnya memuat folder models, folder pages, dan berkas main.dart.
- Apabila AI langsung memberikan semua kode, jangan menyalinnya. Kirim ulang kalimat: “Ikuti format keluaran. Berikan rencana dan daftar berkas saja, tanpa kode.”
- Hasil yang harus diperiksa: proyek dapat dibuka, folder lib terlihat, dan AI belum menulis kode sebelum diminta.



Gambar 5.1 Rencana kerja proyek yang dihasilkan oleh AI

b. Membuat Latihan setState() dengan Bantuan AI

Tahap ini meminta AI membangun halaman latihan setState(), yaitu halaman dengan penghitung pesanan dan sakelar status kantin.

- Buat berkas kosong lib/pages/halaman_set_state.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

```
Lanjutkan ke tahap latihan setState().

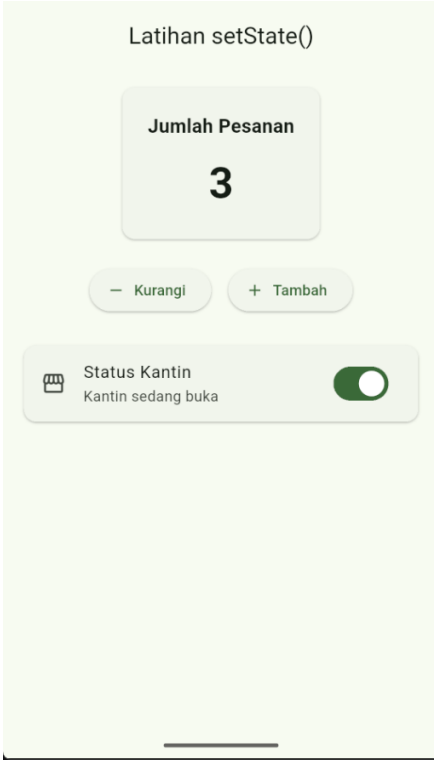
Tugas - lib/pages/halaman_set_state.dart:
1. Buat StatefulWidget bernama HalamanSetState.
2. Sediakan variabel _jumlahPesanan bertipe int mulai dari 0 dan _kantinBuka bertipe bool mulai dari true.
3. Buat fungsi _tambahPesanan yang menambah _jumlahPesanan sebanyak satu melalui setState().
4. Buat fungsi _kurangiPesanan yang mengurangi _jumlahPesanan tetapi tidak boleh menjadi negatif.
5. Buat fungsi _ubahStatusKantin yang mengubah _kantinBuka melalui setState().
6. Tampilkan Scaffold berisi kartu jumlah pesanan, dua tombol (tambah dan kurangi), serta SwitchListTile untuk status kantin.

Batasan: hanya gunakan package:flutter/material.dart. Tampilkan berkas secara lengkap dalam satu blok kode.

Setelah kode, jelaskan mengapa perubahan variabel harus ditulis di dalam setState() dan apa yang terjadi jika baris setState() dihapus.
```

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.

- Bandingkan struktur kode dengan versi manual pada materi di atas. Nama variabel dan fungsi boleh berbeda, tetapi logika penambahan, pengurangan, dan sakelar harus sama.
- Hasil yang harus diperiksa: tombol tambah dan kurangi mengubah angka pada kartu, angka tidak pernah kurang dari nol, dan sakelar mengubah keterangan status kantin.



Gambar 5.2 Halaman latihan setState() hasil AI

c. Memahami Alur setState() dengan Bantuan AI

Tahap ini tidak menghasilkan kode baru. AI diminta menjelaskan alur setState() supaya mahasiswa dapat membandingkan pemahaman sendiri dengan penjelasan AI.

- Kirim prompt berikut pada percakapan yang sama, tanpa meminta AI mengubah kode.

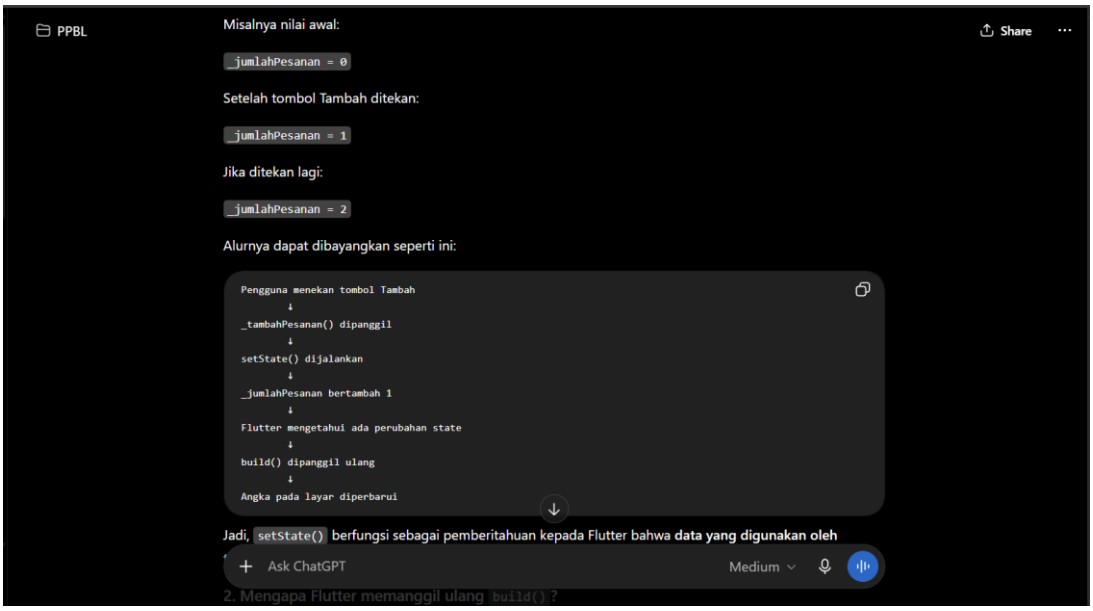
```
Jelaskan alur kerja setState() pada berkas lib/pages/halaman_set_state.dart yang baru dibuat, langkah demi langkah, dengan bahasa mahasiswa pemula.

Untuk setiap langkah, jelaskan:
1. Apa yang terjadi ketika tombol Tambah ditekan.
2. Mengapa Flutter memanggil ulang method build() setelah setState() dipanggil.
3. Bagian mana dari tampilan yang benar-benar berubah dan bagian mana yang tetap sama.
4. Apa yang terjadi jika setState() dipanggil di luar kelas State.

Jangan mengubah kode. Jawab hanya dengan penjelasan.
```

- Tuliskan alur yang dijelaskan AI menggunakan kalimat sendiri, lalu bandingkan dengan diagram alur pada bagian Dasar Teori.
- Jika penjelasan AI berbeda dengan pemahaman sendiri, tandai bagian yang berbeda dan diskusikan pada sesi praktikum.

- Hasil yang harus diperiksa: mahasiswa dapat menjelaskan ulang alur aksi pengguna → setState() → build() → tampilan baru tanpa membaca ulang jawaban AI.



Gambar 5.3 Penjelasan alur setState() oleh AI

d. Merancang Model Keranjang dengan Bantuan AI

Tahap ini merancang struktur data KeranjangModel terlebih dahulu, sebelum menuliskan kode ChangeNotifier secara lengkap pada langkah berikutnya.

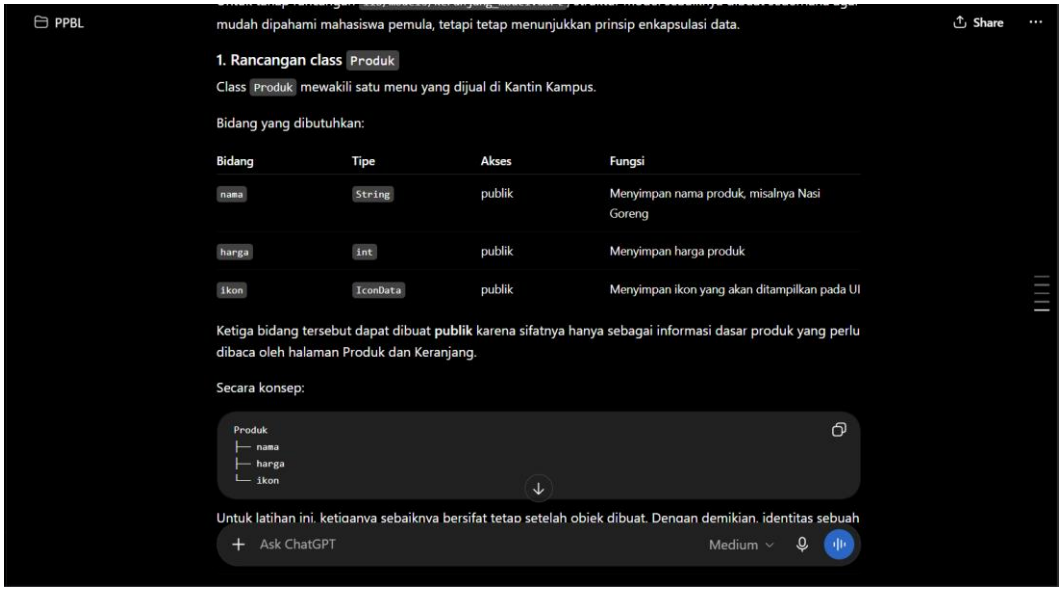
- Buat folder lib/models, lalu kirim prompt berikut.

```
Lanjutkan proyek state_interaksi_ai_nim ke tahap keranjang belanja.

Tugas - rancangan lib/models/keranjang_model.dart:
1. Usulkan struktur class Produk dengan bidang nama, harga, dan ikon.
2. Usulkan struktur class ItemKeranjang yang menyimpan satu Produk dan jumlah.
3. Usulkan struktur class KeranjangModel yang menyimpan daftar ItemKeranjang secara privat.
4. Sebutkan getter yang diperlukan agar UI dapat membaca daftar item, total jumlah barang, dan total harga tanpa mengubah data secara langsung.

Batasan: pada tahap ini jangan menuliskan kode lengkap. Tuliskan hanya rancangan bidang, getter, dan alasan setiap bagian bersifat privat atau publik.
```

- Bandingkan usulan AI dengan kebutuhan pada materi D.d, khususnya alasan _items harus privat.
- Jika AI langsung menuliskan kode lengkap beserta notifyListeners(), simpan dulu jawabannya dan lanjutkan ke prompt pembuatan ChangeNotifier pada langkah berikutnya.
- Hasil yang harus diperiksa: rancangan menyebutkan Produk, ItemKeranjang, KeranjangModel, dan alasan penggunaan getter sebelum kode ditulis.



Gambar 5.4 Rancangan struktur KeranjangModel dari AI

e. Membuat ChangeNotifier dengan Bantuan AI

Tahap ini mengubah rancangan pada langkah sebelumnya menjadi kode ChangeNotifier yang lengkap dan dapat memberi tahu pendengarnya.

- Kirim prompt berikut untuk melengkapi berkas lib/models/keranjang_model.dart.

```
Lengkapi lib/models/keranjang_model.dart berdasarkan rancangan sebelumnya.

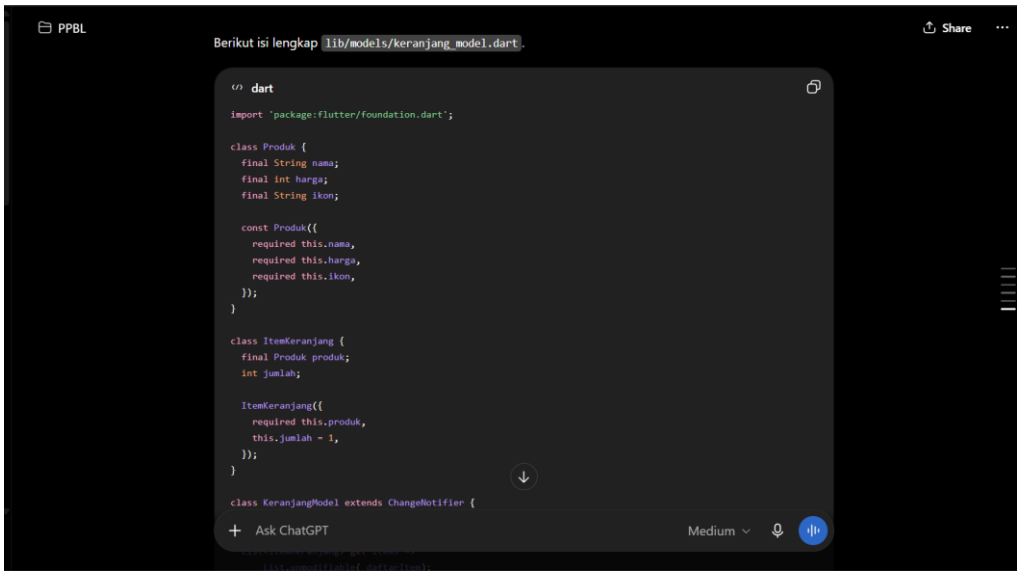
Tugas:
1. Tuliskan class Produk, ItemKeranjang, dan KeranjangModel secara lengkap, dengan KeranjangModel meng-extend ChangeNotifier.
2. Tambahkan metode tambah(Produk produk) yang menambah jumlah jika produk sudah ada, atau menambahkan item baru jika belum ada.
3. Tambahkan metode kurangi(Produk produk) yang mengurangi jumlah, dan menghapus item ketika jumlah mencapai nol.
4. Tambahkan metode kosongkan() yang mengosongkan seluruh isi keranjang.
5. Panggil notifyListeners() pada setiap metode yang benar-benar mengubah data.

Batasan: hanya gunakan package:flutter/foundation.dart. Tampilkan seluruh berkas dalam satu blok kode.

Setelah kode, jelaskan mengapa notifyListeners() harus dipanggil setelah data berubah, bukan sebelum atau tanpa perubahan sama sekali.
```

- Salin kode ke berkas yang sesuai, lalu jalankan flutter analyze untuk memastikan tidak ada galat penulisan.
- Periksa apakah _items tetap privat dan getter yang dihasilkan sesuai dengan rancangan pada langkah sebelumnya.

- Hasil yang harus diperiksa: berkas keranjang_model.dart lengkap, setiap metode yang mengubah data memanggil notifyListeners(), dan tidak ada galat saat flutter analyze dijalankan.



Gambar 5.5 Kode ChangeNotifier KeranjangModel hasil AI

f. Memasang Provider pada Akar Aplikasi dengan Bantuan AI

Tahap ini menempatkan KeranjangModel di atas seluruh halaman yang membutuhkannya melalui ChangeNotifierProvider.

- Kirim prompt berikut untuk memperbarui lib/main.dart.

```
Perbarui lib/main.dart agar KeranjangModel tersedia bagi seluruh aplikasi.

Tugas:
1. Bungkus widget aplikasi dengan ChangeNotifierProvider, dengan create: (context) => KeranjangModel().
2. Pastikan ChangeNotifierProvider berada di atas MaterialApp, bukan di dalamnya.
3. Buat percobaan sederhana: tampilkan Text yang membaca totalItem dari KeranjangModel melalui Consumer, untuk memastikan Provider berhasil dipasang.

Batasan: hanya gunakan package:flutter/material.dart dan package:provider/provider.dart. Tampilkan seluruh berkas dalam satu blok kode.

Setelah kode, jelaskan mengapa ChangeNotifierProvider harus berada di atas widget yang membacanya, bukan di bawah atau sejajar.
```

- Jalankan aplikasi. Jika muncul galat ProviderNotFoundException, minta AI memeriksa kembali posisi ChangeNotifierProvider terhadap MaterialApp.
- Hasil yang harus diperiksa: aplikasi berjalan tanpa galat dan angka totalItem tampil sesuai isi KeranjangModel percobaan.



Gambar 5.6 Uji coba Provider

g. Membuat Halaman Produk dengan Bantuan AI

Halaman produk hanya perlu menambahkan item ke keranjang, sehingga tidak perlu mendengarkan perubahan KeranjangModel.

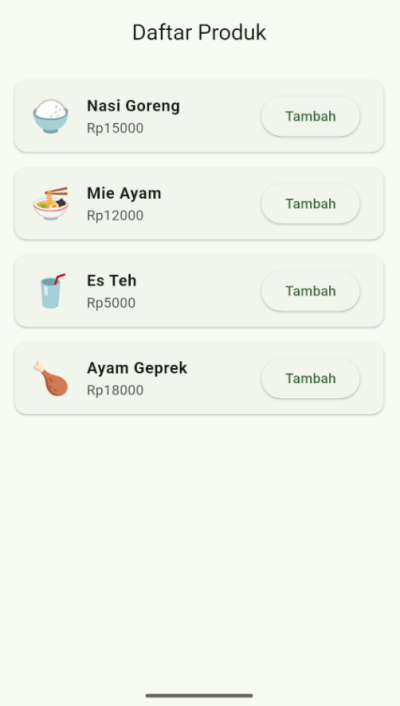
- Kirim prompt berikut untuk membuat lib/pages/halaman_produk.dart.

```
Lanjutkan proyek dengan membuat halaman daftar produk.

Tugas - lib/pages/halaman_produk.dart:
1. Buat StatelessWidget bernama HalamanProduk.
2. Sediakan daftar produk tetap, minimal tiga produk dengan nama, harga, dan ikon berupa emoji.
3. Tampilkan daftar produk memakai ListView.separated, setiap item berupa Card dengan ListTile.
4. Setiap ListTile memiliki tombol Tambah yang memanggil context.read<KeranjangModel>().tambah(produk).

Batasan: gunakan context.read(), bukan context.watch() atau Consumer, karena halaman ini tidak perlu dibangun ulang ketika keranjang berubah. Jelaskan alasannya setelah kode.
```

- Salin kode, jalankan aplikasi, lalu tekan tombol Tambah beberapa kali.
- Hasil yang harus diperiksa: daftar produk tampil lengkap dan tombol Tambah menambah isi keranjang tanpa membangun ulang seluruh daftar produk.



Gambar 5.7 Halaman Produk hasil AI

h. Membuat Halaman Keranjang dengan Bantuan AI

Halaman keranjang harus mengikuti setiap perubahan data, sehingga memakai Consumer agar bagian yang mendengarkan terlihat jelas.

- Kirim prompt berikut untuk membuat lib/pages/halaman_keranjang.dart.

```
Lanjutkan proyek dengan membuat halaman keranjang.

Tugas - lib/pages/halaman_keranjang.dart:
1. Buat StatelessWidget bernama HalamanKeranjang.
2. Bungkus isi halaman dengan Consumer<KeranjangModel>.
3. Jika daftar item kosong, tampilkan empty state berupa ikon keranjang dan teks 'Keranjang masih kosong'.
4. Jika tidak kosong, tampilkan daftar item memakai ListView.separated, setiap item menampilkan nama, jumlah, harga, serta tombol tambah dan kurang yang memanggil metode pada KeranjangModel.
5. Tampilkan total harga di bagian bawah beserta tombol Kosongkan Keranjang.

Batasan: gunakan Consumer, bukan context.watch(), agar bagian yang dibangun ulang terlihat jelas dari kode. Jelaskan setelah kode mengapa Consumer dipilih untuk halaman ini.
```

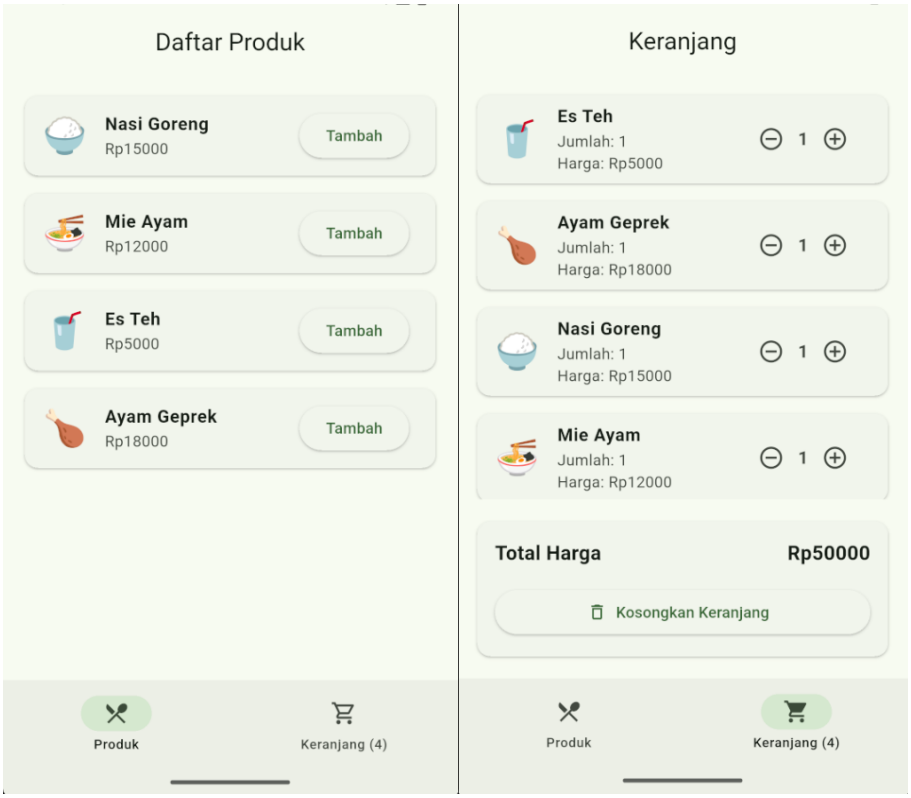
- Salin kode, lalu uji tombol tambah, kurang, dan Kosongkan Keranjang.
- Hasil yang harus diperiksa: halaman keranjang mengikuti setiap perubahan data, total harga selalu sesuai, dan empty state tampil ketika keranjang kosong.
- Tahap terakhir menyatukan halaman Produk dan Keranjang dalam satu kerangka navigasi, kemudian menelusuri galat yang mungkin muncul.
- Kirim prompt berikut untuk melengkapi lib/main.dart versi akhir.

```
Lengkapi lib/main.dart menjadi versi akhir yang menggabungkan halaman Produk dan Keranjang.

Tugas:
1. Gunakan NavigationBar dengan dua tujuan: Produk dan Keranjang.
2. Simpan indeks tujuan terpilih sebagai state lokal memakai setState(), bukan Provider.
3. Tampilkan jumlah totalItem pada judul tujuan Keranjang, dibaca melalui context.select<KeranjangModel, int>.
4. Pertahankan ChangeNotifierProvider yang sudah dipasang pada tahap sebelumnya.

Batasan: hanya satu state yang boleh memakai Provider, yaitu isi keranjang. Indeks navigasi tetap memakai setState(). Jelaskan setelah kode mengapa dua state ini tidak digabung menjadi satu Provider.
```

- Jalankan aplikasi, lalu pindah antara tujuan Produk dan Keranjang.



Gambar 5.8 Aplikasi Kantin Kampus gabungan setState() dan Provider

i. Menggabungkan, Menguji, dan Memperbaiki Galat dengan Bantuan AI

- Jika muncul galat, catat pesan galat lengkap, lalu kirim prompt berikut tanpa meminta AI menulis ulang seluruh berkas.

```
Saya mendapatkan galat berikut saat menjalankan proyek Flutter state_interaksi_ai_nim:

<tempelkan pesan galat lengkap di sini>
```

```
Tugas kamu:
1. Jelaskan arti galat dengan bahasa sederhana.
2. Sebutkan nama berkas dan baris yang paling mungkin menjadi penyebab.
3. Tampilkan perbaikan minimal, hanya bagian kode yang berubah.
4. Jelaskan mengapa perbaikan tersebut menyelesaikan galat.

Jangan menulis ulang seluruh berkas dan jangan mengubah fitur lain.
```

- Terapkan perbaikan secara manual, jalankan flutter analyze, lalu flutter run kembali.
- Hasil yang harus diperiksa: navigasi antara Produk dan Keranjang berjalan, jumlah pada tujuan Keranjang selalu sesuai, dan tidak ada galat saat aplikasi dijalankan.

j. Mengevaluasi Jawaban AI

Tahap penutup ini melatih Saudara menilai kode dari AI, bukan sekadar menyalinnya.

- Tanpa membuka kembali percakapan AI, jawab pertanyaan berikut menggunakan kata-kata sendiri: kapan sebaiknya memakai setState() dan kapan memakai Provider; mengapa notifyListeners() harus dipanggil setelah data berubah; serta mengapa context.read() tidak membuat widget membangun ulang tampilannya.
- Pilih satu bagian kode dari AI yang belum sepenuhnya dipahami, lalu kirim prompt berikut tanpa meminta AI mengubah kodenya.

```
Jelaskan potongan kode Flutter berikut baris demi baris untuk mahasiswa pemula. Jangan mengubah kodenya.

<tempelkan potongan kode yang belum dipahami>

Untuk setiap bagian, jelaskan:
1. Apa tugasnya.
2. Data apa yang masuk dan keluar.
3. Apa yang terjadi jika bagian itu dihapus.
4. Apakah bagian itu berhubungan dengan setState() atau dengan Provider.
```

G. Tugas Praktikum

Soal 1: Menambahkan Manajemen State pada Aplikasi Layanan Warga “Nusantara Cerdas Mobile”

Aplikasi layanan warga Nusantara Cerdas Mobile yang dibangun pada Modul II dinilai berhasil oleh Dinas Komunikasi dan Informatika. Warga kini dapat berpindah antara tujuan Beranda, Layanan, dan Warga tanpa harus menggulir satu halaman panjang, membuka menu pendukung melalui panel samping, serta menikmati tampilan yang menyesuaikan diri antara layar sempit dan layar lebar di lobi balai kota. Akan tetapi, seluruh data pada aplikasi tersebut masih berupa nilai tetap yang ditulis langsung di dalam kode, sehingga setiap interaksi warga belum benar-benar tersimpan. Kepala dinas menerima banyak keluhan bahwa warga harus mengingat

sendiri layanan mana yang pernah mereka cari di antara tiga bidang layanan, dan tidak ada cara untuk mengajukan beberapa layanan sekaligus dalam satu kali kunjungan ke balai kota.

Kepala dinas kemudian meminta tiga peningkatan pada purwarupa yang sudah ada. Pertama, warga harus dapat menandai layanan pada tujuan Layanan sebagai favorit agar mudah ditemukan kembali dari tujuan Warga, tanpa memandang tab bidang Perizinan, Kesehatan, atau Transportasi tempat layanan tersebut berada. Kedua, warga harus dapat memasukkan beberapa layanan ke dalam satu keranjang pengajuan sebelum mengirimkannya sekaligus ke dinas terkait, dan jumlah layanan yang sedang diajukan harus tetap terlihat pada ikon tujuan Warga meskipun warga sedang membuka tujuan Beranda atau Layanan. Ketiga, tombol Ajukan Permohonan pada halaman rincian layanan harus menampilkan tanda sedang mengirim sesaat sebelum permohonan benar-benar tercatat, agar warga tidak menekan tombol tersebut berulang kali karena mengira aplikasi tidak merespons. Petugas balai kota menegaskan bahwa ketiga peningkatan ini tidak boleh mengubah kerangka navigasi yang sudah berjalan. Sebagai anggota tim pengembang, tugas Anda adalah melanjutkan purwarupa tersebut dengan menambahkan pengelolaan state yang sesuai.

Ketentuan Pengerjaan

- Lanjutkan proyek nusantara_cerdas_nav_nim yang kerangka navigasinya sudah dibuat pada Modul II. Jika belum pernah dikerjakan, buat ulang kerangka navigasi dasarnya (NavigationBar, NavigationDrawer, tab navigation, dan NavigationRail adaptif) sebelum menambahkan state.
- Buat berkas lib/models/favorit_model.dart berisi class FavoritModel yang meng-extend ChangeNotifier, menyimpan kumpulan nama layanan favorit, serta metode tandai(String namaLayanan) dan batalTandai(String namaLayanan) yang memanggil notifyListeners().
- Buat berkas lib/models/pengajuan_model.dart berisi class PengajuanModel yang meng-extend ChangeNotifier, menyimpan daftar layanan yang sedang diajukan beserta getter totalPengajuan.
- Pasang FavoritModel dan PengajuanModel melalui MultiProvider di akar aplikasi, di atas MaterialApp yang sudah memuat named routes dari Modul II.
- Pada tujuan Layanan, setiap kartu layanan diberi ikon bintang yang memanggil context.read<FavoritModel>() untuk menandai atau membatalkan tanda, sedangkan warna ikon mengikuti status favorit melalui context.watch<FavoritModel>().
- Tambahkan bagian Layanan Favorit pada tujuan Warga yang menggunakan Consumer<FavoritModel> agar daftar layanan favorit tampil reaktif, tanpa memandang tab bidang asal layanan tersebut.
- Pada halaman rincian layanan, tombol Ajukan Permohonan menyimpan status sedang mengirim sebagai state lokal melalui setState(), menampilkan indikator singkat, kemudian menambahkan layanan ke PengajuanModel setelah proses selesai, sambil tetap mengirim nilai balik melalui Navigator.pop(context, nilai) seperti pada Modul II.
- Tampilkan badge jumlah pengajuan pada ikon tujuan Warga menggunakan context.select<PengajuanModel, int> agar hanya bagian badge yang dibangun ulang ketika totalPengajuan berubah, tanpa membangun ulang NavigationBar maupun NavigationRail.
- Pastikan penempatan Provider tidak menimbulkan ProviderNotFoundException pada layar sempit maupun layar lebar yang sudah dibuat pada Modul II.

- Jalankan aplikasi pada emulator atau perangkat fisik dan pada peramban Google Chrome, tandai beberapa layanan sebagai favorit, ajukan beberapa permohonan, lalu ambil tangkapan layar setiap tahap pada tampilan layar sempit dan layar lebar.

Luaran yang Dikumpulkan

- Kode program lanjutan pada repositori Git.
- Laporan dalam format PDF yang dikumpulkan melalui LMS.
- Tangkapan layar layanan sebelum dan setelah ditandai favorit, bagian Layanan Favorit pada tujuan Warga yang menampilkan daftar reaktif, badge jumlah pengajuan yang berubah, serta indikator sedang mengirim saat tombol Ajukan Permohonan ditekan.
- Penjelasan singkat pada laporan mengenai alasan pemilihan setState() untuk status sedang mengirim dan ChangeNotifier/Provider untuk data favorit serta pengajuan, termasuk alasan pemilihan context.watch(), context.read(), atau context.select() pada setiap bagian.
- Berkas laporan diberi nama Modul3_NIM_NamaLengkap.pdf dan diunggah melalui LMS sesuai tenggat yang ditentukan.

Soal 2: Menambahkan Manajemen State pada Rancangan Tugas Akhir Saudara

Ketentuan Pengerjaan

- Lanjutkan proyek tubes_namaaplikasi_nim yang kerangka navigasinya sudah dibuat pada Soal 2 Modul II.
- Identifikasi minimal satu data yang bersifat lokal dan hanya penting bagi satu halaman atau satu widget, misalnya status tampil/sembunyi, indeks tab lokal, atau status memuat, lalu kelola data tersebut dengan setState().
- Identifikasi minimal satu data yang dipakai bersama oleh dua halaman atau lebih, misalnya daftar favorit, keranjang, atau status masuk pengguna, lalu buat model yang meng-extend ChangeNotifier untuk data tersebut.
- Sediakan model melalui ChangeNotifierProvider atau MultiProvider di akar aplikasi, di atas widget navigasi yang sudah dibuat pada Modul II.
- Tampilkan data dari Provider tersebut pada minimal dua halaman berbeda, masing-masing memakai context.watch(), context.read(), atau context.select() sesuai kebutuhan halaman tersebut.
- Data yang dikelola boleh sementara atau contoh sederhana, tetapi perubahan data harus terlihat nyata pada tampilan, bukan nilai tetap.
- Pastikan aplikasi tetap berjalan tanpa ProviderNotFoundException pada seluruh halaman yang mengakses Provider.

Luaran yang Dikumpulkan

- Kode program pada repositori Git.
- Tangkapan layar setiap halaman sebelum dan setelah state berubah.
- Laporan Modul3_Soal2_NIM_NamaLengkap.pdf diunggah melalui LMS, menjelaskan data mana yang memakai setState() dan mana yang memakai Provider, beserta alasannya.

H. Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. **Halaman Sampul (Cover)**
   Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.
2. **Tujuan Praktikum**
   Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.
3. **Alat dan Bahan**
   Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya Flutter SDK, Dart SDK, Android Studio, Visual Studio Code, dan Git.
4. **Langkah-Langkah Praktikum**
   Menguraikan tahapan pengerjaan secara runtut mulai dari pembuatan proyek, pembuatan stateless widget, stateful widget, penyusunan layout dan styling, hingga penggabungan pada main.dart. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.
5. **Hasil dan Dokumentasi Program**
   Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya tampilan awal dan tampilan setelah terjadi perubahan state. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.
6. **Analisis dan Pembahasan**
   Membahas alasan pemilihan stateless widget dan stateful widget pada masing-masing bagian aplikasi, kendala atau galat yang ditemukan beserta cara penyelesaiannya, serta keterkaitan aplikasi dengan pilar smart city yang diangkat.
7. **Kesimpulan**
   Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.
8. **Lampiran berisi Link Repositori GitHub**
   Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu, memuat kode program lengkap.
9. **Presentasi Hasil Praktikum**
   Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.